

# Deep Learning

## 09 Training DNNs II

Dr. Konda Reddy Mopuri  
Dept. of Artificial Intelligence  
IIT Hyderabad  
Jan-May 2026

# So far

- Basic Neuron Models, MLP

# So far

- Basic Neuron Models, MLP
- UAT, Backprop

# So far

- Basic Neuron Models, MLP
- UAT, Backprop
- Training DNNs - SGD variations

# Today

- Training DNNs - II
  - Data Preprocessing
  - Parameter Initialization
  - Regularization in DNNs

# 1. Data preprocessing

- Why to preprocess the data?

# 1. Data preprocessing

- Why to preprocess the data?
- Raw data usually suffers incompleteness, inconsistencies, noise, etc.

# 1. Data preprocessing

- Why to preprocess the data?
- Ensure data quality

# 1. Data preprocessing

- Why to preprocess the data?
- Ensure data quality
- Missing values, noise need to be handled

# 1. Data preprocessing

- Why to preprocess the data?
- Matching the learning algorithm requirements

# 1. Data preprocessing

- Why to preprocess the data?
- Matching the learning algorithm requirements
- They may be sensitive to the format and scale of the data

# 1. Data preprocessing

- Why to preprocess the data?
- Matching the learning algorithm requirements
- They may be sensitive to the format and scale of the data
- Gradient descent based models can be dominated by features with large numerical ranges

# 1. Data preprocessing

- Why to preprocess the data?
- Improving model performance

# 1. Data preprocessing

- Why to preprocess the data?
- Improving model performance
- Removing irrelevant features or reducing the impact of outliers

# 1. Data preprocessing

- Why to preprocess the data?
- Improving model performance
- Removing irrelevant features or reducing the impact of outliers
- Improving the efficiency (e.g., dimensionality reduction)

# 1. Data preprocessing

- How to preprocess the data?
- General approach is to ensure the data is “scaled, stationary, and representative”

# 1. Data preprocessing

- How to preprocess the data?
- Scaling/Standardization/Normalization:  $z = \frac{x-\mu}{\sigma}$

# 1. Data preprocessing

- How to preprocess the data?
- Scaling/Standardization/Normalization:  $z = \frac{x-\mu}{\sigma}$
- Other normalizations also observed (e.g., Min-Max)

# 1. Data preprocessing

- How to preprocess the data?
- Beyond One-Hot Encoding

# 1. Data preprocessing

- How to preprocess the data?
- Beyond One-Hot Encoding
- Great for categories with low cardinality

# 1. Data preprocessing

- How to preprocess the data?
- Beyond One-Hot Encoding
- Great for categories with low cardinality
- For high-cardinality tasks, embedding layer can be learned

# 1. Data preprocessing

- How to preprocess the data?
- Beyond One-Hot Encoding
- Great for categories with low cardinality
- For high-cardinality tasks, embedding layer can be learned
- Learns a low-dim, dense representation that are semantic

# 1. Data preprocessing

- How to preprocess the data?
- Task dependent

# 1. Data preprocessing

- How to preprocess the data?
- Task dependent
- Zero-centering in CV, tokenization (keeping more of the original context) in NLP, etc.

## 2. Weight Initialization

- Some optimization algorithms are not iterative (no problem with them)

## 2. Weight Initialization

- Some optimization algorithms are not iterative (no problem with them)
- Some are iterative, like DNNs and hence need an initialization

## 2. Weight Initialization

- Some optimization algorithms are not iterative (no problem with them)
- Some are iterative, like DNNs and hence need an initialization
- It can determine if the training can converge (fast/at all)

## 2. Weight Initialization

- Modern initialization strategies are simple and heuristic

## 2. Weight Initialization

- Modern initialization strategies are simple and heuristic
- This is because DNN optimization is not yet well understood

## 2. Weight Initialization

- Most strategies are based on achieving some 'nice' properties during the initialization

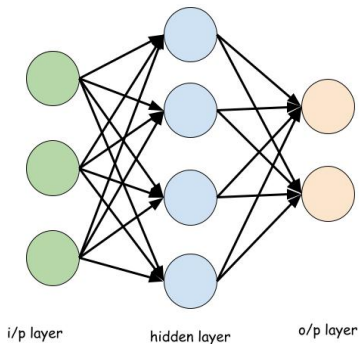
## 2. Weight Initialization

- Most strategies are based on achieving some 'nice' properties during the initialization
- But, we may not know which of these properties hold after the learning begins

## 2. Weight Initialization

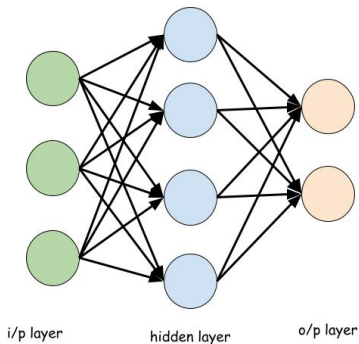
- One property with certainty is that initialization need to break the 'symmetry'

## 2. Weight Initialization



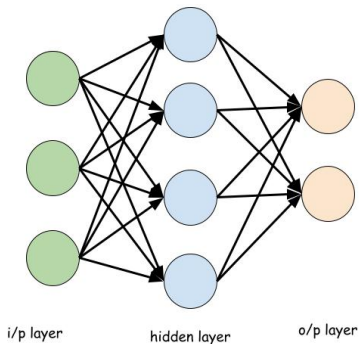
- What if all the parameters are initialized to zero?

## 2. Weight Initialization



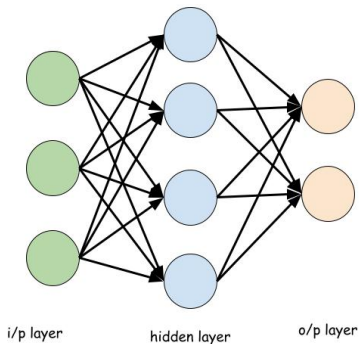
- What if all the parameters are initialized to zero?
- Or, a different constant?

## 2. Weight Initialization



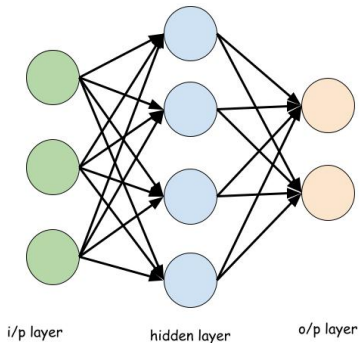
- What if all the parameters are initialized to zero?
- Or, a different constant?
- Leads to a failure mode (often known as the **'symmetry'** problem)

## 2. Weight Initialization



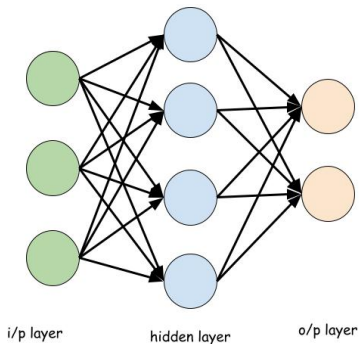
- What if all the parameters are initialized to zero?
- Or, a different constant?
- Leads to a failure mode (often known as the '**symmetry**' problem)
- Hence, we need different values as weights!

## 2. Weight Initialization



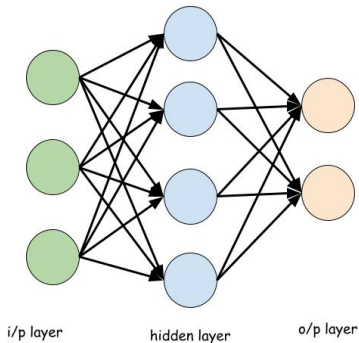
- Is it good enough to have different parameters?

## 2. Weight Initialization



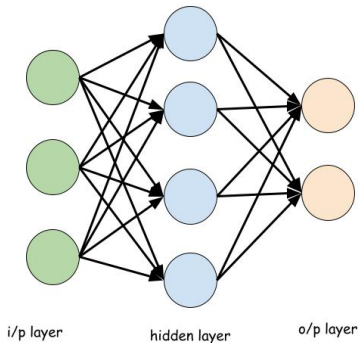
- Is it good enough to have different parameters?
- Large weights  $\rightarrow$  **exploding gradients**

## 2. Weight Initialization



- Is it good enough to have different parameters?
- Large weights → **exploding gradients**
- Small ones → **vanishing gradients**

## 2. Weight Initialization



- Is it good enough to have different parameters?
- Large weights → **exploding gradients**
- Small ones → **vanishing gradients**
- Different weights → different o/p range of the neurons

## 2. Weight Initialization

- How about randomly initializing?

$W = 0.001 * \text{np.random.randn}(d_l, d_{l-1})$

---

Figure credits: Dr Justin Johnson, U Michigan

## 2. Weight Initialization

- How about randomly initializing?  
 $W = 0.001 * \text{np.random.randn}(d_l, d_{l-1})$
- Maybe, okay for the shallow nets

---

Figure credits: Dr Justin Johnson, U Michigan

## 2. Weight Initialization

- How about randomly initializing?  
 $W = 0.001 * \text{np.random.randn}(d_l, d_{l-1})$
- Maybe, okay for the shallow nets
- However, the dynamic range of the activations at later layers goes on shrinking  $\rightarrow$  activations tend to zero at deeper layers (e.g. 6 layer MLP with a tanh nonlinearity)

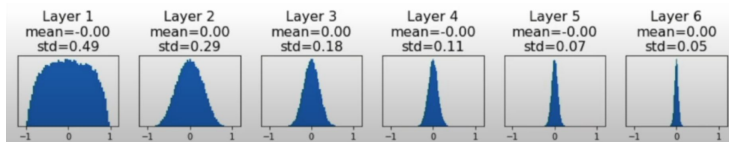
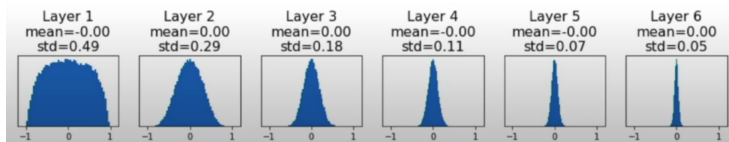


Figure credits: Dr Justin Johnson, U Michigan

## 2. Weight Initialization

- How about randomly initializing?  
 $W = 0.001 * \text{np.random.randn}(d_l, d_{l-1})$
- Maybe, okay for the shallow nets
- However, the dynamic range of the activations at later layers goes on shrinking  $\rightarrow$  activations tend to zero at deeper layers (e.g. 6 layer MLP with a tanh nonlinearity)



- All zero gradients, no learning!

Figure credits: Dr Justin Johnson, U Michigan

## 2a. Xavier Initialization

- $W = \text{np.random.randn}(d_l, d_{l-1}) / \text{np.sqrt}(d_{l-1})$

---

Figure credits: Dr Justin Johnson, U Michigan

## 2a. Xavier Initialization

- $W = \text{np.random.randn}(d_l, d_{l-1}) / \text{np.sqrt}(d_{l-1})$

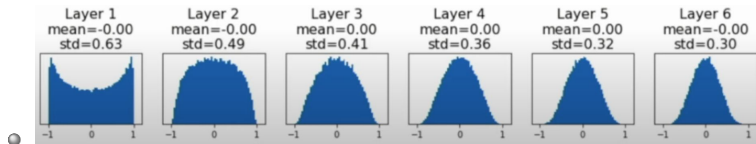


Figure credits: Dr Justin Johnson, U Michigan

## 2a. Xavier/Glorot Initialization (2010)



భారతీయ టెక్నాలజీ హైదరాబాద్  
भारतीय प्रौद्योगिकी संस्थान हैदराबाद  
Indian Institute of Technology Hyderabad

- Idea: Keep the variance of the activations and that of gradients same across all the layers

## 2a. Xavier/Glorot Initialization (2010)



- Idea: Keep the variance of the activations and that of gradients same across all the layers
- → Signal and Gradient flow without fading away or blowing up!

## 2a. Xavier Initialization

- Let's look at the forward pass

## 2a. Xavier Initialization

- Let's look at the forward pass
- Consider a single layer,  $y = Wx$ , i.e.  $y_i = \sum_{j=1}^{d_{l-1}} x_j \cdot w_j$

## 2a. Xavier Initialization

- Let's look at the forward pass
- Consider a single layer,  $y = Wx$ , i.e.  $y_i = \sum_{j=1}^{d_{l-1}} x_j \cdot w_j$
- $\text{var}(y_i) = d_{l-1} \cdot \text{var}(x_i \cdot w_i)$  (Assuming  $w_i$  and  $x_i$  are i.i.d)

## 2a. Xavier Initialization

- Let's look at the forward pass
- Consider a single layer,  $y = Wx$ , i.e.  $y_i = \sum_{j=1}^{d_{l-1}} x_j \cdot w_j$
- $\text{var}(y_i) = d_{l-1} \cdot \text{var}(x_i \cdot w_i)$  (Assuming  $w_i$  and  $x_i$  are i.i.d)
- $\text{var}(y_i) = d_{l-1} \cdot \left( E(x_i^2) \cdot E(w_i^2) - E(x_i)^2 \cdot E(w_i)^2 \right)$  (Assuming  $x$  and  $w$  are independent and zero mean)

## 2a. Xavier Initialization

- Let's look at the forward pass
- Consider a single layer,  $y = Wx$ , i.e.  $y_i = \sum_{j=1}^{d_{l-1}} x_j \cdot w_j$
- $\text{var}(y_i) = d_{l-1} \cdot \text{var}(x_i \cdot w_i)$  (Assuming  $w_i$  and  $x_i$  are i.i.d)
- $\text{var}(y_i) = d_{l-1} \cdot \left( E(x_i^2) \cdot E(w_i^2) - E(x_i)^2 \cdot E(w_i)^2 \right)$  (Assuming  $x$  and  $w$  are independent and zero mean)
- $\text{var}(y_i) = d_{l-1} \cdot \text{var}(x_i) \cdot \text{var}(w_i)$  Assuming ( $x_i$  and  $w_i$  are zero-mean)

## 2a. Xavier Initialization

- Let's look at the forward pass
- Consider a single layer,  $y = Wx$ , i.e.  $y_i = \sum_{j=1}^{d_{l-1}} x_j \cdot w_j$
- $\text{var}(y_i) = d_{l-1} \cdot \text{var}(x_i \cdot w_i)$  (Assuming  $w_i$  and  $x_i$  are i.i.d)
- $\text{var}(y_i) = d_{l-1} \cdot \left( E(x_i^2) \cdot E(w_i^2) - E(x_i)^2 \cdot E(w_i)^2 \right)$  (Assuming  $x$  and  $w$  are independent and zero mean)
- $\text{var}(y_i) = d_{l-1} \cdot \text{var}(x_i) \cdot \text{var}(w_i)$  Assuming ( $x_i$  and  $w_i$  are zero-mean)
- $\rightarrow \text{var}(w_i) = \frac{1}{d_{l-1}}$

## 2a. Xavier Initialization

- Similar logic applies to the gradients moving backward during the backprop forward pass

## 2a. Xavier Initialization

- Similar logic applies to the gradients moving backward during the backprop forward pass
- $\rightarrow \text{var}(w_i) = \frac{1}{d_l}$

## 2a. Xavier Initialization

- Similar logic applies to the gradients moving backward during the backprop forward pass
- $\rightarrow \text{var}(w_i) = \frac{1}{d_l}$
- Xavier and Bengio proposed using the average of these two constraints as a compromise  $\text{var}(w_i) = \frac{2}{d_l + d_{l-1}}$

## 2a. Xavier Initialization

- Similar logic applies to the gradients moving backward during the backprop forward pass
- $\rightarrow \text{var}(w_i) = \frac{1}{d_l}$
- Xavier and Bengio proposed using the average of these two constraints as a compromise  $\text{var}(w_i) = \frac{2}{d_l + d_{l-1}}$
- If we use Uniform distribution  $U(-a, a)$ ,  $a = \sqrt{\frac{6}{d_l + d_{l-1}}}$

## 2b. Weight Initialization with ReLU activations

- Kaiming He or MSRA initialization

---

Figure credits: Dr Justin Johnson

## 2b. Weight Initialization with ReLU activations

- Kaiming He or MSRA initialization
- $\text{std} = \sqrt{2/d_{l-1}}$

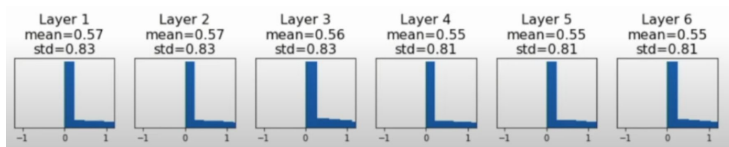


Figure credits: Dr Justin Johnson

## 2c. Weight Initialization: Residual Networks (2015)

- MSRA initialization:  
 $\text{Var}(F(x)+x) > \text{Var}(x)$

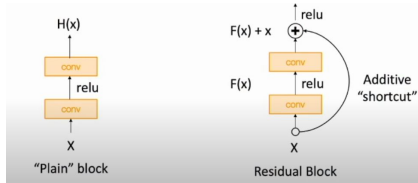
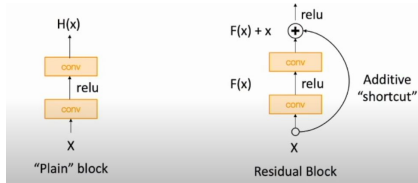


Figure credits: Dr. Justin Johnson

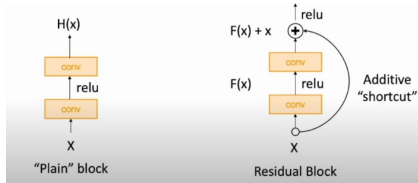
## 2c. Weight Initialization: Residual Networks (2015)



- MSRA initialization:  
 $\text{Var}(F(x)+x) > \text{Var}(x)$
- Variance grows!

Figure credits: Dr. Justin Johnson

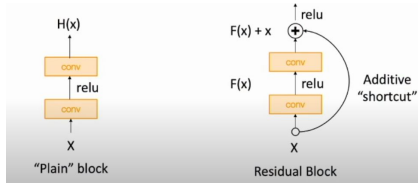
## 2c. Weight Initialization: Residual Networks (2015)



- MSRA initialization:  
 $\text{Var}(F(x)+x) > \text{Var}(x)$
- Variance grows!
- Solution: Initialize the scale parameter ( $\gamma$ ) of the last BN layer with zero

Figure credits: Dr. Justin Johnson

## 2c. Weight Initialization: Residual Networks (2015)



- MSRA initialization:  
 $\text{Var}(F(x)+x) > \text{Var}(x)$
- Variance grows!
- Solution: Initialize the scale parameter ( $\gamma$ ) of the last BN layer with zero
- The entire residual branch outputs zero (initially!)  $\rightarrow$   
 $\text{Var}(x+F(x)) = \text{Var}(x)$

Figure credits: Dr. Justin Johnson

## 2c. Weight Initialization in latest models

- Most modern LLMs don't follow Xavier/He initialization

## 2c. Weight Initialization in latest models

- Most modern LLMs don't follow Xavier/He initialization
- They follow 'Truncated Normal Distribution' with  $\mu = 0$  and  $\sigma = 0.02$

## 2c. Weight Initialization in latest models

- Most modern LLMs don't follow Xavier/He initialization
- They follow 'Truncated Normal Distribution' with  $\mu = 0$  and  $\sigma = 0.02$
- Empirical heuristic that keeps the weights small enough to avoid saturation of activations

## 2c. Weight Initialization in latest models

- Most modern LLMs don't follow Xavier/He initialization
- They follow 'Truncated Normal Distribution' with  $\mu = 0$  and  $\sigma = 0.02$
- Empirical heuristic that keeps the weights small enough to avoid saturation of activations
- They handle the variance growth due to skip connections by scaling the  $\sigma$  by depth (in the o/p projection layer):  $\sigma_{\text{res}} = \frac{0.02}{\sqrt{(2.N)}}$

# 3. Deep Regularization

- ① Most of the regularization techniques trade increased bias for decreased variance

# 3. Deep Regularization

- ① Most of the regularization techniques trade increased bias for decreased variance
- ② It has to be profitable!

# 3. Deep Regularization

- ① Most often the best-fitting model is a large model that has been appropriately regularized

# 3. Deep Regularization

- Parameter Norm penalties ( $l_2$ ,  $l_1$ , etc.)
- Dataset Augmentation
- Noise Robustness
- Semi-Supervised Learning
- Multi-Task Learning (Parameter sharing)
- Sparse Representation
- Dropout
- etc.

## 3a. Parameter Norm Penalties

- ① For neural networks, typically only the weights of the affine transformations are regularized leaving the biases unregularized

## 3a. Parameter Norm Penalties

- ① For neural networks, typically only the weights of the affine transformations are regularized leaving the biases unregularized
- ② Bias controls only a single variable as opposed to weight which connects two

## 3a. Parameter Norm Penalties

- ① For neural networks, typically only the weights of the affine transformations are regularized leaving the biases unregularized
- ② Bias controls only a single variable as opposed to weight which connects two
- ③ Regularizing biases may induce underfitting

## 3a. Parameter Norm Penalties

①  $L_2$  parameter regularization:  $\tilde{\mathcal{J}} = \frac{\alpha}{2} w^T w + \mathcal{J}(w; X, y)$

## 3a. Parameter Norm Penalties

- ①  $L_2$  parameter regularization:  $\tilde{\mathcal{J}} = \frac{\alpha}{2} w^T w + \mathcal{J}(w; X, y)$
- ②  $L_1$  regularization:  $\tilde{\mathcal{J}} = \alpha |w|_1 + \mathcal{J}(w; X, y)$

## 3a. Parameter Norm Penalties

- ①  $L_2$  parameter regularization:  $\tilde{\mathcal{J}} = \frac{\alpha}{2} w^T w + \mathcal{J}(w; X, y)$
- ②  $L_1$  regularization:  $\tilde{\mathcal{J}} = \alpha |w|_1 + \mathcal{J}(w; X, y)$
- ③ Norm penalties induce different desired behaviors based on the exact penalty imposed

## 3b. Dataset Augmentation

- ① Bestway to make ML model generalize better is to train with more data

## 3b. Dataset Augmentation

- ① Bestway to make ML model generalize better is to train with more data
- ② In practice training data is limited

## 3b. Dataset Augmentation

- ① Bestway to make ML model generalize better is to train with more data
- ② In practice training data is limited
- ③ Create fake data and add it to the training data, called Dataset augmentation

## 3b. Dataset Augmentation

- ① Easier for classification

## 3b. Dataset Augmentation

- ① Easier for classification
- ② Difficult for density estimation task (unless we have solved the estimation problem)

## 3b. Dataset Augmentation

- ① Has been particularly effective for specific classification problems such as object recognition

## 3b. Dataset Augmentation

- ① Has been particularly effective for specific classification problems such as object recognition
- ② Operations such as translation by few pixels, rotating slightly, adding mild noise, etc. greatly improve generalization

## 3b. Dataset Augmentation

- ① Has been particularly effective for specific classification problems such as object recognition
- ② Operations such as translation by few pixels, rotating slightly, adding mild noise, etc. greatly improve generalization
- ③ Hand-designed augmentations in some domains can result in dramatic improvements

## 3b. Dataset Augmentation

- ① Has been particularly effective for specific classification problems such as object recognition
- ② Operations such as translation by few pixels, rotating slightly, adding mild noise, etc. greatly improve generalization
- ③ Hand-designed augmentations in some domains can result in dramatic improvements
- ④ Should restrict to label preserving transformations

## 3c. Multi-Task Learning

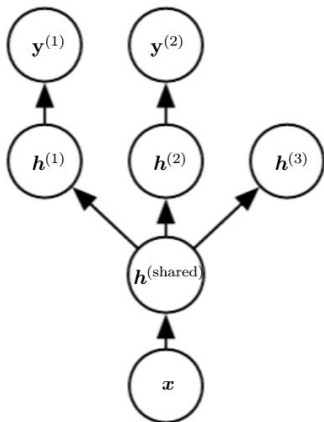
- ① Improves generalization by collecting samples arising out of multiple tasks

## 3c. Multi-Task Learning

- ① Improves generalization by collecting samples arising out of multiple tasks
- ② Similar to additional data samples, multi-task samples also put more pressure on the parameters of the shared layers to be 'better'

## 3c. Multi-Task Learning

- ① Improves generalization by collecting samples arising out of multiple tasks
- ② Similar to additional data samples, multi-task samples also put more pressure on the parameters of the shared layers to be better



## 3d. Dropout

- ① Key ideas and contributions in DL have been to engineer architectures for making them easier to train

## 3d. Dropout

- ① Key ideas and contributions in DL have been to engineer architectures for making them easier to train
- ② Dropout is one such ('deep') regularization technique (Srivastava et al. 2014)

## 3d. Dropout

- ① During the forward pass, some of the units are randomly 'zeroed' out (neurons are removed)

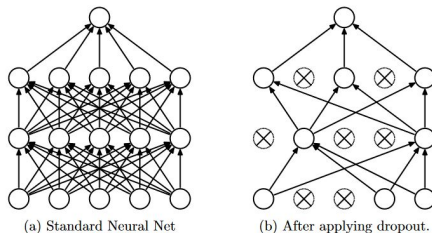


Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

Figure from Srivastava et al. 2014

## 3d. Dropout

- ① During the forward pass, some of the units are randomly 'zeroed' out (neurons are removed)
- ② Dropped units are randomly selected in each layer independent of others

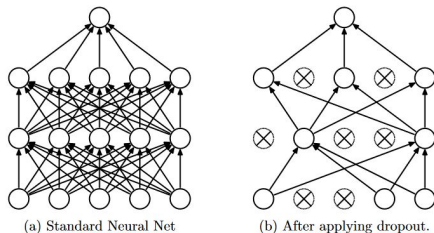


Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

Figure from Srivastava et al. 2014

## 3d. Dropout

- ① During the forward pass, some of the units are randomly 'zeroed' out (neurons are removed)
- ② Dropped units are randomly selected in each layer independent of others
- ③ Resulting network has a different architecture

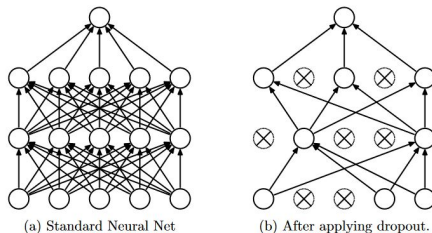


Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

Figure from Srivastava et al. 2014

## 3d. Dropout

- ① During the forward pass, some of the units are randomly 'zeroed' out (neurons are removed)
- ② Dropped units are randomly selected in each layer independent of others
- ③ Resulting network has a different architecture
- ④ Backpropagation happens through the remaining activations

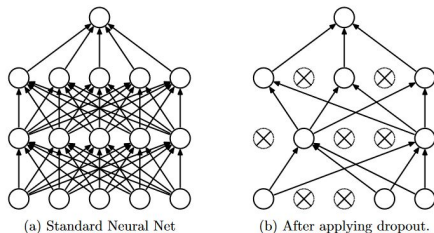


Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

Figure from Srivastava et al. 2014

## 3d. Dropout: Interpretation

- ① Improves independence between the units (prevents co-adaptation of the units in the network)

## 3d. Dropout: Interpretation

- ① Improves independence between the units (prevents co-adaptation of the units in the network)
- ② Distributes the representation among all the units (forces the network to learn redundancy)

## 3d. Dropout

- ① We will decide on which units/layers to use dropout, and with what probability  $p$  units are dropped.

## 3d. Dropout

- ① We will decide on which units/layers to use dropout, and with what probability  $p$  units are dropped.
- ② For each sample, as many Bernoulli variables as units are sampled independently for dropping the units.

## 3d. Dropout: Another Interpretation

- ① Results in a large ensemble of networks (with shared parameters)

### 3d. Dropout: Another Interpretation

- ① Results in a large ensemble of networks (with shared parameters)
- ② Every possible binary mask results in a member of the ensemble

### 3d. Dropout: Another Interpretation

- ① Results in a large ensemble of networks (with shared parameters)
- ② Every possible binary mask results in a member of the ensemble
- ③ E.g. a dense layer with 10 units has  $2^{10}$  masks!

## 3d. Dropout: test time

- ① Which model from the ensemble to use?  
 $y = f(x, w, m)$  ( $m$  is the chosen binary mask)

## 3d. Dropout: test time

- ① Which model from the ensemble to use?  
 $y = f(x, w, m)$  ( $m$  is the chosen binary mask)
- ② How about taking the opinion of all the experts? → 'average out' and make the o/p deterministic

### 3d. Dropout: test time

- ① Which model from the ensemble to use?  
 $y = f(x, w, m)$  ( $m$  is the chosen binary mask)
- ② How about taking the opinion of all the experts? → ‘average out’ and make the o/p deterministic
- ③  $y = \mathbb{E}_m[f(x, w, m)] = \sum_m p(m) \cdot f(x, w, m)$

### 3d. Dropout: test time

- ① Which model from the ensemble to use?  
 $y = f(x, w, m)$  ( $m$  is the chosen binary mask)
- ② How about taking the opinion of all the experts?  $\rightarrow$  'average out' and make the o/p deterministic
- ③  $y = \mathbb{E}_m[f(x, w, m)] = \sum_m p(m) \cdot f(x, w, m)$
- ④ Leads to dropping no unit but multiply the activations with the probability of retaining

## 3d. Dropout

- ① Which layers to regularize with the Dropout?

## 3d. Dropout

- ① Which layers to regularize with the Dropout?
- ② More parameters are the dense layers → usually applied there

## 3d. Dropout

- ① Which layers to regularize with the Dropout?
- ② More parameters are the dense layers → usually applied there
- ③ Not much used after ResNets!

## 3e. Batch Normalization (BN)

- ① Gradient Descent converges faster with feature scaling ( $x \leftarrow \frac{x-\mu}{\sigma}$ )

## 3e. Batch Normalization (BN)

- ① Gradient Descent converges faster with feature scaling ( $x \leftarrow \frac{x-\mu}{\sigma}$ )
- ② Batch Normalization (BN) is a normalization method for intermediate layers of NNs  $\rightarrow$  performs whitening to the intermediate layer activations

## 3e. Batch Normalization (BN)

**Input:** Values of  $x$  over a mini-batch:  $\mathcal{B} = \{x_{1\dots m}\}$ ;

Parameters to be learned:  $\gamma, \beta$

**Output:**  $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

$\gamma$  and  $\beta$  are learn-able parameters

## 3e. Batch Normalization (BN)

- 1 Originally introduced to handle the internal covariate shift (ICS)

## 3e. Batch Normalization (BN)

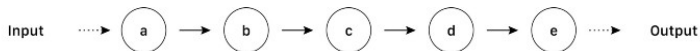
- ① Originally introduced to handle the internal covariate shift (ICS)
- ② BN makes the activation of each neuron to be Gaussian distributed

## 3e. Batch Normalization (BN)

- ① Originally introduced to handle the internal covariate shift (ICS)
- ② BN makes the activation of each neuron to be Gaussian distributed
- ③ ICS is undesirable because the layers need to adapt to the new distribution of activations
- ④ With BN, it is reduced to new pair of parameters, but the distribution remains Gaussian

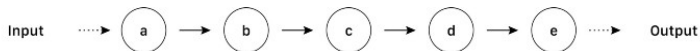
## 3e. Batch Normalization (BN)

- ① Mitigates interdependency between hidden layers during training



## 3e. Batch Normalization (BN)

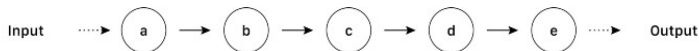
- ① Mitigates interdependency between hidden layers during training



- ②  $\partial(a) = \partial(b) \cdot \partial(c) \cdot \partial(d) \cdot \partial(e)$

## 3e. Batch Normalization (BN)

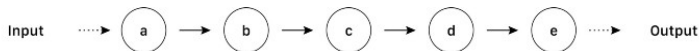
- ① Mitigates interdependency between hidden layers during training



- ②  $\partial(a) = \partial(b) \cdot \partial(c) \cdot \partial(d) \cdot \partial(e)$
- ③ if we want to adjust the input distribution of a specific hidden unit, we need to consider the whole sequence of layers (w/o BN)

## 3e. Batch Normalization (BN)

- ① Mitigates interdependency between hidden layers during training



- ②  $\partial(a) = \partial(b) \cdot \partial(c) \cdot \partial(d) \cdot \partial(e)$
- ③ if we want to adjust the input distribution of a specific hidden unit, we need to consider the whole sequence of layers (w/o BN)
- ④ BN acts like a valve which holds back the flow, and allows its regulation using  $\beta$  and  $\gamma$

## 3e. Batch Normalization (BN)

- ① Reduces training time (less ICS)

## 3e. Batch Normalization (BN)

- ① Reduces training time (less ICS)
- ② Reduces the demand for additional regularizers (Batch statistics)

## 3e. Batch Normalization (BN)

- ① Reduces training time (less ICS)
- ② Reduces the demand for additional regularizers (Batch statistics)
- ③ Allows higher learning rates (less danger of vanishing/exploding gradients)

# Regularization: General idea

- ① Make all the assumptions explicit  $\rightarrow$  appropriate inductive biases.